

Authority-Aligned Linearization for Rescue-Service Incident Response over Intermittent Edge Networks

Anonymous Author(s)

Anonymous Affiliation

Submitted to NCA 2026 (double-blind review)

Abstract—Rescue-service edge systems must keep working during responder and backhaul partitions without letting concurrent replicas issue conflicting authority-bearing incident decisions. We present authority-aligned linearization (AAL), the consistency discipline implemented by the AAL prototype. AAL places the incident-journal linearization point at the edge node that currently holds operational command authority. Non-authority data (reference state, field submissions, materialized views) remains locally available and is forwarded eventually. Command transfer uses fenced promotion, not leader election. We contrast AAL with consensus-based state-machine replication, multi-primary replication, and conflict-free replicated data type (CRDT)-style merge. TLA+ and Hypothesis models check the single-authority and fenced-promotion invariants. A Docker Compose prototype measures field-submission propagation, wide-area network recovery, throughput, duplicate replay, and stale-epoch rejection on the service path. Partitions on either responder or backhaul links do not block command-edge progress, and stale-epoch traffic cannot poison the incident journal. The fence stale-rejection path is measured to be $2.5\times$ cheaper than a successful commit, by construction. The cost is no automatic failover; promotion is operator-initiated.

Index Terms—authority-aligned linearization, network architectures and protocols, edge computing, intermittent connectivity, distributed systems, incident response, prototype evaluation

I. INTRODUCTION

Public-safety information systems must remain operational when commercial connectivity fails [1]–[3]. Rescue-service vehicles operate up to several hours from regional dispatch centres in terrain where wireless connectivity alternates between full wide-area coverage, mesh-only operation, and total isolation [4]–[6]. Disasters routinely degrade or destroy commercial telecom infrastructure precisely when coordination demand is highest [7], as documented in the 2018 Swedish forest-fire crisis [8]. Vehicles must continue acting on stale or partial information without external coordination. Field submissions entered on tablets must survive hours without connectivity. In command-hierarchical environments the consequences of information loss are wrong assignments, missed hazards, and conflicting orders [9].

The operational constraints in this paper are grounded in public Swedish rescue-service guidance: Räddningstjänsten Storgöteborg’s incident-planning and municipal rescue-plan guidance [10], [11], which treats incident-planning information as an ongoing, partly secrecy-restricted process handled in operational mobile IT support rather than a static paper artifact. Independent academic work on Swedish rescue-service information-systems practice corroborates these end-

user requirements and information-flow shortcomings [12], [13]. From these public sources we derive five constraints. The system must operate without connection to any other tier. Tablets need durable forward-only field submissions. Incident-state decisions need clear command authority. The network design must avoid cross-vehicle broadcast hazards. The deployment must reuse existing rescue-service certificate and identity infrastructure. Section II-C separates these externally motivated requirements from design assumptions and integration choices.

This paper asks: *how should authority-bearing data be synchronized across intermittently connected edge tiers, when the operational model designates a single principal as authoritative?* The answer is authority-aligned linearization (AAL); a discipline that places the incident-journal linearization point at the command edge, the vehicle edge node that currently holds operational command authority. AAL is the prototype system used to evaluate AAL. It combines core-to-edge reference-data distribution, responder outboxes, a single incident journal, and fenced promotion. The contribution is not a new consensus or CRDT mechanism; it is an empirical and architectural characterization of where the linearization boundary should sit for this class of rescue-service workflow. We make three contributions:

- 1) an empirical characterization of the authority-aligned linearization boundary, including a measured comparison with a Hanssen-style operation-based CRDT-LWW baseline on the same queue-depth workload shape and an explicitly scoped outage-duration comparison;
- 2) a fenced-promotion protocol that keeps the writer of record aligned with operational command authority, specified in TLA+ and mirrored by Hypothesis state-machine tests, with a service-path check that stale-epoch traffic is rejected at the command edge;
- 3) a Docker Compose and *tc-netem* prototype study of responder-to-command propagation, command-to-core recovery, incident-journal linearization, idempotent replay, and the limits of the current implementation.

The remainder of the paper motivates the requirements and architecture, presents the synchronization protocol, describes the prototype, reports the emulation and bounded safety results, and closes with scope and limitations.

II. PROBLEM ANALYSIS

A. Operational Setting

The target setting is municipal rescue-service incident response. A regional core supports dispatch, reference data, and backhaul; vehicle edge nodes run near the incident site; field tablets attach to a local vehicle and act as leaf clients. Connectivity is intermittent: vehicles may move between wide-area connectivity, mesh-only contact, and total isolation [4], [5]. One vehicle edge node holds the command role for the incident; we call it the command edge. The role can move to another vehicle, but only through an explicit human decision, and the protocol must respect that operational authority. Figure 1 illustrates the actors and the three connectivity regimes.

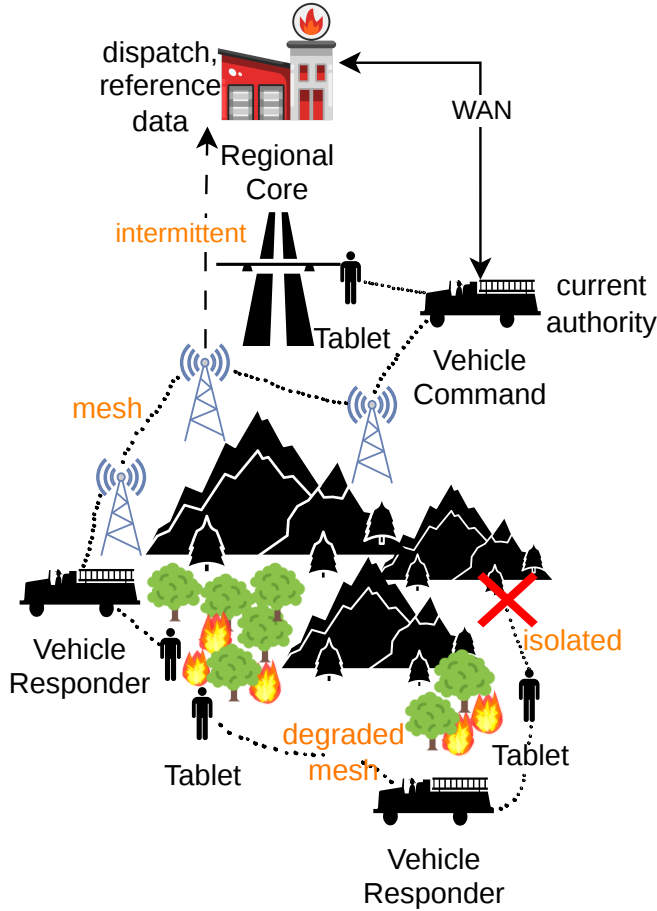


Fig. 1. Operational setting. A regional core supports dispatch and reference data; vehicle edge nodes run near the incident site; tablets attach to a local vehicle. One vehicle holds command authority at any instant. Vehicles simultaneously occupy different connectivity regimes: wide-area backhaul, intermittent backhaul with mesh transit, or total isolation.

B. Position Relative to Consensus, CRDTs, and Emergency Networking

AAL recombines well-known mechanisms (single-leader replication, transactional outbox [14], idempotency keys, manual fencing) under a domain-specific rule; the incident-journal linearization point is the command edge, set by an external

operational protocol rather than chosen by liveness-driven leader election. The incident-state entity owns its decision history; external actors interact through forward-only events rather than mutate state directly [15]. The novelty is therefore in (i) which operations are placed under that linearization point and which are not, and (ii) the fenced-promotion rule that preserves single-authority across command changes. The mechanisms themselves are not claimed as novel.

Existing systems solve adjacent parts of the problem—offline-first stores, conflict-free replicated data types (CRDTs), consensus logs, and emergency/edge networking—but none, by itself, states which rescue-service actor may issue non-commutative command decisions during a partition.

Offline-first and local-first systems prioritize local availability and continued operation under poor connectivity [14], [16]–[18], and CRDT and strong-eventual-consistency work—including richer local-first safety models and Hanssen’s CRDT replication for emergency management, our closest baseline—provides convergence for mergeable replicated state [19]–[25]. Consensus, shared-log, and chain-replication systems provide replicated agreement and ordered logs under their stated failure models [26]–[32]. These help responders keep working or order writes, but none decides *which actor* may issue an authority-bearing decision. AAL keeps authority-bearing events under the incident-journal sequencer, treats outbox entries as pending until command acceptance, and assigns weaker contracts to non-authority data.

Closest in setting is emergency and edge networking. CoNICE [33] reaches agreement among infrastructure-less peers under intermittent connectivity using One-Third-Rule consensus, evaluated on shared map-pin updates; it is the right tool when no peer is privileged, whereas rescue-service incident command has a designated commander at every moment and the network is not allowed to vote on incident decisions. Public-safety networking and edge/fog research place resilient transport and storage near responders [5], [34]–[36]. We are not aware of prior work that explicitly uses operational command authority as the linearization boundary while assigning weaker contracts to non-authority incident data; AAL aligns the incident-journal linearization point with the current operational command authority and uses fenced promotion rather than leader election for command transfer.

C. Requirements and Assumptions

Public rescue-service guidance and studies of Swedish rescue-service information systems motivate the requirements used here [10]–[13]. The requirements are separate from the architecture. In particular, attributable command authority does not logically require a single-writer database design. Aligning the incident-journal linearization boundary with command authority is the design choice made in this paper.

The externally motivated requirements are:

- **R1 (Offline operation)** Each tier must keep working from previously synchronized data without continuous connectivity.

TABLE I
SYNCHRONIZATION CONTRACTS BY STATE CLASS.

State	Authority/source	Synchronization contract
Reference data	regional core	One-way core-to-edge replication; eventual convergence.
Immutable artifacts	publisher / core	Manifest, package, or hash validation where implemented.
Field observations	tablet / responder	Locally durable outbox; at-least-once forwarding; idempotent accept.
Command decisions	command edge	Single-writer rule applies only here; total order by <i>event_seq</i> .
Materialized state	journal	Derived and rebuildable; not authoritative.
Audit events	edge / core	Eventual forwarding; validation incomplete.

- **R2 (Durable forward-only submission)** Field submissions entered on tablets must survive vehicle restart and prolonged partition.
- **R3 (Attributable command authority)** Resource assignments, status transitions, and hazard annotations must be attributable to one operational command authority at any instant.

The design assumes that an incident has one current command authority. The command role is fixed at dispatch and changes only through an explicit operator decision. The deployment constraints are separate: the mesh should act as routed transit rather than a stretched user network, and the system must reuse the rescue-service identity infrastructure. Their architectural consequences—edge cache and local services, tablet/edge outbox and retry, a single incident-journal writer, mesh-as-transit routing, and mTLS/CA identity integration—are developed in §III.

III. ARCHITECTURE

A. State Classes and Consistency Contracts

AAL does not apply one consistency model to all data. The single-writer rule applies only to authority-bearing incident-journal entries. This split is a RedBlue-style [37] consistency partition with authority-bearing decisions in the strong set and forward-only field submissions, cached reference data, and materialized state in the weaker classes. Table I summarizes the main state classes.

Master and reference data are owned by the regional core and cached at edges. Their partition behavior is stale-but-available local read. Field submissions are durable at the responder first. They become authoritative only after command acceptance. Authority-bearing incident events are sequenced by the command edge with a command-assigned *event_seq*. Materialized state is derived from the append-only incident journal.

Artifacts and attachments use immutable or manifest-based distribution. Audit records and command-to-core backhaul are forwarded eventually. Promotion and fencing metadata are part of the command-authority protocol. The current evidence for promotion/fencing is model-level; Section IV gives the protocol rule.

B. Tiered Edge Topology

The architecture follows from these constraints. The regional core provides master/reference data and receives backhaul. Vehicle edges provide local services during disconnection. Responder vehicles queue submissions durably. The command edge owns the append-only incident journal. Tablets are leaf clients and do not participate directly in command sequencing.

The mesh is transit, not the consistency mechanism. User VLANs are not extended across vehicles. Application services communicate between edge nodes over routed links in the target deployment.

IV. SYNCHRONIZATION PROTOCOL

The main identifiers are *incident_id*, a client-supplied *client_event_id*, and a command-assigned *event_seq*. The promotion model also uses a command epoch or equivalent term.

A. Linearization Boundary (M1)

M1 defines the state-contract distinction introduced in Section III-A: which operations require incident-journal linearization, and which use weaker contracts. The incident journal is authoritative for incident decisions; responder outboxes, tablet caches, and materialized views are derived or pending state. The single-writer rule applies only to authority-bearing incident-journal entries.

B. Incident Journal Sequencing (M2)

M2 specifies how the command edge linearizes authority-bearing operations into a totally ordered journal. The command edge accepts a *client_event_id* once and assigns *event_seq* atomically with journal insertion, mapping duplicates to the existing journal entry rather than resequencing them. Consumers apply committed entries in *event_seq* order; materialized state is derived from the journal and can be rebuilt.

C. Durable Outbox and Idempotent Submission (M3)

M3 specifies the responder-side path for non-authority data: locally durable, idempotent, and eventually forwarded to the incident journal. Figure 2 shows the steady-state field-submission path. The boundary is between local durability and authoritative command commit. A responder may acknowledge local acceptance after the outbox insert. It marks an item forwarded only after incident-journal acceptance or duplicate recognition.

The M3 contract has three parts:

- 1) Each submission has a globally unique idempotency key, *client_event_id*, scoped by the incident and originating client identity.
- 2) A responder accepts a submission only after the event is stored in its local outbox.
- 3) A responder may mark an outbox item forwarded only after the command edge acknowledges durable incident-journal acceptance or duplicate recognition.

Outbox delivery is at least once and is deduplicated at the command edge; there is no claim of causal ordering across independent responder outboxes before command assignment.

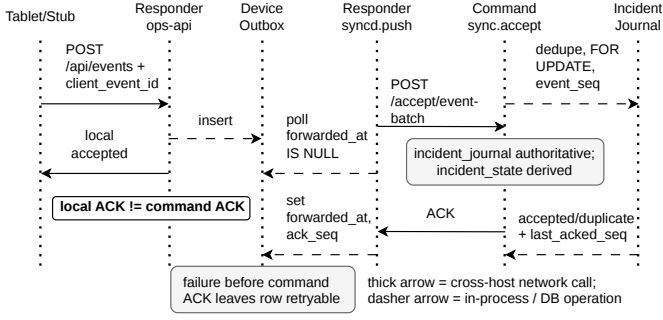


Fig. 2. Responder outbox protocol. The local acknowledgement (ACK) to the tablet is not the command acknowledgement: a submission is acknowledged locally after durable outbox insertion, but the row is only marked forwarded after incident-journal acceptance. The incident journal assigns the authoritative sequence number, and duplicate delivery is handled by *client_event_id*. Line weight encodes hop kind: thick arrows cross hosts (HTTP/mTLS), thin arrows are in-process or DB operations.

D. Fenced Promotion (M4)

M4 specifies fenced promotion, the mechanism used to transfer command authority promotion without forking the journal. The design does not provide autonomous failover. An operator confirms fenced promotion, and fencing prevents stale command authority from continuing. Figure 3 summarizes the intended rule. Each command tenure has a monotonically increasing epoch or equivalent term. Promotion emits an explicit record. Stale command traffic must be rejected once a newer epoch is known. Section VI-B reports the service-path measurement of this rule.

This strict rule sacrifices availability when the previous command cannot be fenced or its status cannot be established; Section VI-B checks this boundary by comparing strict promotion with a weak variant.

E. Partition Behavior

If a responder is cut off from command, it can keep using cached data and store new submissions in its local outbox, but those submissions are not authoritative until the command edge accepts them and assigns *event_seq*. If command is cut off from the regional core, command-edge incident work continues and the core catches up later from the committed incident journal. If command must move to another vehicle, AAL does not elect a replacement automatically: an operator must promote the new command edge and prevent stale command authority from continuing to write, otherwise promotion is blocked.

V. PROTOTYPE IMPLEMENTATION

The repository contains a prototype service path for the core and edge tiers, plus deployment scaffolding for the target vehicle environment. Python services target Python 3.12 and use FastAPI, Pydantic models, asyncpg, and uvicorn. Database schemas are managed through numbered SQL migrations. Docker Compose files instantiate the emulated core, command edge, and responder edges used in Section VI; a separate *baseline/crdt* Compose stack provides the CRDT-LWW

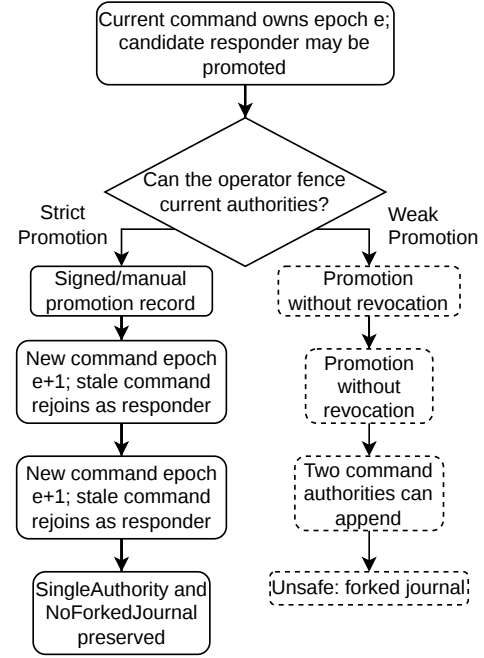


Fig. 3. Fenced promotion preserves the single-writer invariant. Left (single-border): with fencing, the stale command is forced to rejoin as a responder. Right (double-border): without fencing, two command authorities can append concurrently, producing a forked journal. The unsafe branch appears as a 3-state TLA+ trace for split authority and a 5-state trace for a forked journal; durable epoch storage and stale-epoch rejection are implemented in the prototype and measured in section VI-B.

comparison point. Table II maps the protocol concepts from Section IV to the repository artefacts that realise them and records the evidence boundary per concept.

The evaluated path is narrower than the full target deployment. The *sync-api* provides a synthetic incident bootstrap endpoint and accepts incident-journal backfill. The edge *ops-api* persists field submissions into *outbox.device_outbox* with *client_event_id* idempotency. The *syncd* responder path forwards unacknowledged outbox entries to the command edge; the command-edge *syncd* path assigns *event_seq* inside a database transaction and deduplicates duplicate *client_event_id* values. The command-to-core forwarder backfills committed incident-journal entries to the core. These are the paths exercised by the evaluation.

Authentication in the target deployment is layered: a VPN overlay protects vehicle-to-core traffic, Nginx terminates TLS/mTLS and passes certificate-derived identity to backend services, and tablets authenticate to their local vehicle using enrolled certificates. In the reported emulation these controls are deployment design, not measured mechanisms: traffic uses plain HTTP on a Docker bridge network and the Python tablet stub supplies synthetic device and user identifiers.

VI. EMULATION & EVALUATION

A. Methodology and setup

The evaluation follows the AAL contract. V-B tests the single-writer boundary across command transitions and its

TABLE II
MAPPING FROM PROTOCOL CONCEPTS TO PROTOTYPE ARTIFACTS.

Concept	Prototype artifact	Status
<i>M1–M2: Linearization boundary and incident-journal sequencing</i>		
Incident journal and event_seq	edge / syncd / src / accept . py; incident . journal; incident.state	Implemented; evaluated on the command-edge syncd accept path.
Command-edge accept during responder partition	evaluate-command-local-partition.py (in scripts/); /accept/event-batch	Evaluated for the direct command-edge syncd accept path during responder isolation; tablet-facing command-edge ops-api not exercised.
<i>M3: Durable outbox and idempotency</i>		
Responder durable outbox	edge / ops - api / src / routes / api . py; outbox . device _ outbox; edge/syncd/src/push.py	Implemented; evaluated for responder forwarding.
client_event_id idempotency	edge/db/migrations/004_idempotency_and_sync_state.sql; accept.py	Implemented; evaluated by duplicate replay (§VI-D).
<i>M4: Fenced promotion</i>		
Promotion and fencing	edge/db/migrations/005_command_epoch.sql; edge/syncd / src / accept . py; promote - responder . sh, evaluate - fenced - promotion.py (in scripts/); formal / tla / RescueOIS . tla; formal/python/tests/test_no_two_writers.py	Durable command-epoch table and stale-epoch rejection implemented at service level; matching TLA+/Hypothesis model checks; measured in §VI-B.
<i>Supporting infrastructure (not authority-aligned)</i>		
Core bootstrap and backhaul	core / sync - api; core / db / migrations / 003_incident_tables.sql	Synthetic bootstrap and incident-journal backfill; evaluated in Docker.
Vehicle edge roles	edge/docker-compose.yml; edge/syncd/src/main.py; EDGE_ROLE	Command and responder edges in Compose.
Incident-journal forwarding to core	edge/syncd/src/forward.py; core / sync - api / src / routes / sync . py; sync.state	Implemented; evaluated for WAN recovery/backfill.
Tablet client	scripts/tablet-stub.py; tablet/	Python stub in evaluation; Android scaffold only.
Multi-condition netem evaluation	scripts / evaluate - netem - propagation . py; edge/syncd/Dockerfile tc tooling	Implemented; measured under two link profiles in §VI-D.
Transit and security controls	docs / deployment / mesh - commissioning . md; core / wireguard/; edge/wireguard/	Target-deployment documentation and configuration; physical mesh and security overhead are out of scope for the reported evaluation.

cost. V-C tests ordinary authority-bearing writes under responder isolation and characterises the serializer. V-D tests the durable non-authority path under replay, restart, and mesh-grade link impairment. V-E asks whether a CRDT-LWW baseline can preserve R3 on the same workload shape. V-F measures support-path backhaul after WAN heal. V-B–V-D exercise M1–M4 of §IV; V-E–V-F delimit what AAL does not promise.

Measurements come from a single-host Docker Compose emulation: one regional core, one command edge, and three responder-role edge nodes on a shared Docker bridge. The propagation, recovery, and throughput cells are characterised through a single responder path; the additional responders

TABLE III
SUMMARY OF SERVICE-PATH EMULATION MEASUREMENTS.

Path / cell	Result	Interpretation
<i>Responder-to-command propagation</i>		
queue depth = 1 ($n = 30$)	median 710.1 ms; p95 832.1 ms	outbox-to-command path
queue depth = 10 ($n = 30$)	median 639.4 ms; p95 719.5 ms	outbox-to-command path
queue depth = 100 ($n = 30$)	median 931.5 ms; p95 1045.5 ms	outbox-to-command path
<i>Command-to-core WAN recovery</i>		
partition = 1 s ($n = 30$)	median 0.75 s; p95 0.92 s	support-path backhaul, not command authority
partition = 10 s ($n = 30$)	median 0.91 s; p95 1.04 s	support-path backhaul, not command authority
partition = 60 s ($n = 30$)	median 0.23 s; p95 1.05 s	support-path backhaul, not command authority
<i>Command journal serialization</i>		
direct /accept ($n = 29$)	median 356.3 events/s; p95 491.7	linearization point: command accept + journal write only
end-to-end via outbox ($n = 29$)	median 159.4 events/s; p95 163.9	full responder outbox → push-batch → command path
<i>Command commit during responder isolation</i>		
direct command accept path ($n = 20$)	20/20 commits; seq. 1–20; 0 duplicate seqs.	direct /accept/event-batch path; median/p95 latency 5.76/7.32 ms

populate the topology rather than the measured path, so the reported latencies are per-path, not aggregate. The command-to-core WAN partition is injected surgically with *tc* on the command syncd: only traffic to and from the core sync-api is dropped, so the command edge keeps reaching its local Postgres and every responder throughout the outage—this is what lets §VI-F measure backhaul recovery in isolation rather than a full re-attach of the command node. Responder isolation, where required, detaches a single responder edge from the bridge. The impairment cells use two *tc-netem* profiles informed by published mesh-network measurements [4], [34]: stressed (80 ms $\pm$ 30 ms delay, 2 % loss) and degraded (200 ms $\pm$ 80 ms delay, 8 % loss).

Parameter choices are fixed for single-host reproducibility rather than tuned. Each cell repeats $n = 30$ unless noted—small enough to keep the matrix runnable on one host, large enough for stable median and p95 estimates, both reported. Propagation queue depths of 1, 10, and 100 isolate per-event cost, exercise a typical batch, and stress the polling/batching path. WAN partition durations of 1, 10, and 60 s span short, medium, and long backhaul outages. Throughput uses 1000 concurrent submissions on two paths: direct /accept/event-batch posts saturate the journal serializer, and responder /api/events posts measure end-to-end M3 delivery. The idempotency cell replays 50 unique events five times each. The TLA+ bounds (§VI-B) use the largest state spaces that complete in minutes on the same host. Table III summarises the primary numbers.

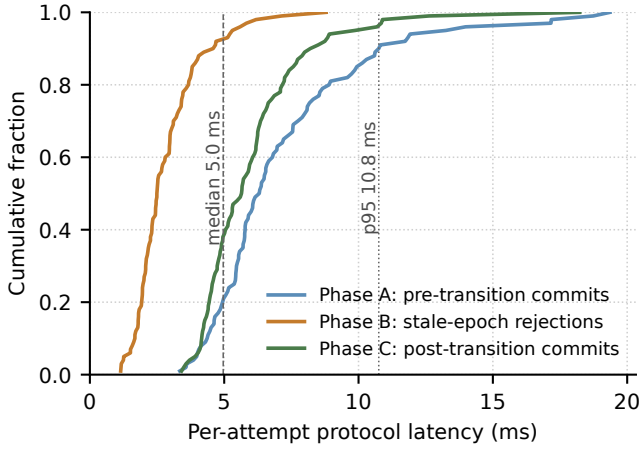


Fig. 4. Fenced-promotion protocol cost by phase. Phase A is pre-transition commits at epoch e , phase B is stale-epoch rejections at the boundary, phase C is post-transition commits at epoch $e + 1$. $n = 300$ events (100 per phase). The cost shown is the protocol path only; the operator-confirm step is excluded by design.

B. Fenced promotion rejects stale epochs cheaply

Fenced promotion is measured first because a failed fence can split authority. The TLA+ and Hypothesis models establish bounded safety invariants; the service-path run shows stale-epoch traffic is rejected before any incident-journal work. Both commit and rejection paths add only single-digit millisecond protocol cost, while operator confirmation remains a seconds-scale wall-clock cost by design.

The protocol cost of fenced promotion (Figure 4) is single-digit milliseconds across all three phases of an active transition, measured over $n = 300$ events (100 per phase). On the commit paths, phase A (pre-transition) has median 6.3 ms (p95 14.0 ms), and phase C (post-transition) has median 5.7 ms (p95 10.7 ms). Phase B, the stale-rejection path, is $2.5\times$ cheaper than the commit paths (median 2.5 ms, p95 5.6 ms) by construction: the fence rejects stale traffic before it reaches the incident-journal write. The saving comes from skipping the write for non-authoritative traffic, not from adding a faster check on the write path. Operator confirmation sits in the critical path by design, bounded below by deliberation under stress on the order of seconds; the protocol contribution is two to four orders of magnitude smaller than that floor. Promotion’s wall-clock is therefore an operator cost, not a network cost.

In five live service-path runs, phase A commits 20/20 events at epoch 1, phase B returns 20/20 HTTP 409 rejections for the stale epoch and adds zero incident-journal rows, and phase C commits 20/20 events at epoch 2. The incident journal contains 200 rows total, split 100/100 between epochs, with no *event_seq* value appearing under both.

Table IV contrasts the cost of verifying the strict variant with the cost of falsifying the weak variant. The strict configuration ($V = 3$, $J = 7$, $C = 7$, $P = 3$) verifies six safety invariants over 38.56M distinct states in 13 minutes. Each weak variant

TABLE IV
FALSIFICATION-COST ASYMMETRY. THE STRICT VARIANT VERIFIES SIX SAFETY INVARIANTS OVER TENS OF MILLIONS OF DISTINCT STATES; EACH WEAK VARIANT PRODUCES A COUNTEREXAMPLE IN HUNDREDS TO THOUSANDS OF DISTINCT STATES UNDER A SECOND OF WALL-CLOCK.

Variant	Outcome	States	Time
strict	all six invariants pass	38.56 M	13.0 min
weak	<i>SingleAuthority</i> violated	185	0.85 s
weak	<i>EpochAuthorityCoupling</i> violated	144	0.77 s
weak	<i>NoForkedJournal</i> violated	1.0 k	0.83 s

fails in under a second of wall-clock; counterexample traces have length 3, 3, and 5 for *SingleAuthority*, *EpochAuthorityCoupling*, and *NoForkedJournal*, found after exploring 185, 144, and 1,015 distinct states respectively. A Hypothesis state-machine test mirrors this distinction in Python over 2000 examples of length 50 with no strict violation and an on-demand weak counterexample. Without the fence, single-authority fails immediately.

C. Authority-bearing writes are independent of responder reachability

After V-B, the next question is whether ordinary authority-bearing writes commit while a responder is unreachable; otherwise R3 would depend on connectivity rather than the protocol. With one responder isolated for ten seconds, 20 single-event POSTs to */accept/event-batch* commit 20/20 (HTTP 200), assign contiguous non-duplicate *event_seq* values, and have median/p95 latency 5.8/7.3 ms ($n = 20$). The endpoint requires a current *X-Command-Epoch*; stale or absent epochs are rejected before journal work. Responder reachability is not in the commit path.

Saturation throughput at the linearization point—direct concurrent POSTs to */accept/event-batch*—is 356.3 events/s ($n = 29$ after warm-up exclusion; p95 491.7). This isolates the prototype serializer: same-incident writes use `SELECT ... FOR UPDATE` over an `asyncpg` pool capped at eight connections. End-to-end M3 throughput is reported separately in §VI-D.

D. Field submissions propagate under realistic intermittence

V-B and V-C cover authority-bearing writes. V-D covers M3: responder submissions must reach the journal durably, without duplicate journal rows, under retry, restart, and mesh-grade impairment.

Propagation latency is 710.1 ms at queue depth 1 and 639.4 ms at queue depth 10, then rises to 931.5 ms at queue depth 100. The floor comes from the responder push loop: `POLL_INTERVAL_S = 0.25`, with four-poll sleep after empty results. The queue-depth-100 rise follows `BATCH_SIZE = 50`: 100 events require two batches separated by the 250 ms post-success sleep, plus local submission and TCP setup; the measured delta over queue depth 10 is 292.1 ms.

The duplicate-replay scenario issues 50 unique *client_event_id* values and replays each five times; across $n = 30$ runs the journal contains exactly 50 rows. Forwarding is at-least-once and deduplicated at the journal, so submission is effectively exactly-once at the linearization point.

Durability across process restart is tested directly. Fifty submissions enter the responder outbox, responder Postgres is power-cycled before forwarding, and the stack recovers. All 50 survive, forward with no duplicates and contiguous *event_seq*, and reach journal consistency about 2.6 s after recovery. This is the runtime witness for R2.

End-to-end submission throughput from responder ops-api to journal commit is 159.4 events/s ($n = 29$ after warm-up exclusion; p95 163.9). It includes durable outbox insert, push batching, polling, and the M1/M2 commit path; it is distinct from the 356.3 events/s linearization-point figure in §VI-C.

Under the same *tc-netem* profiles, each cell runs 20 measurements at queue depths 1 and 10; all 80 measurements complete with no drops. Median/p95 latencies in ms are: stressed qd-1 1215/1494, stressed qd-10 2860/3355, degraded qd-1 3647/5329, degraded qd-10 8104/10895. The heaviest tail is degraded qd-10 (median 8.1 s, p95 10.9 s).

E. CRDT semantics fail for authority-bearing data

V-D shows the non-authority path holds under impairment and restart. V-E asks the inverse: if the linearization boundary were removed and incident data were mergeable by default, would R3 still hold?

The CRDT-LWW baseline is four FastAPI replicas using Hanssen’s operation-based Last-Writer-Wins Element Set with physical-clock timestamps [25]. It preserves local acceptance but loses authority-bearing conflicts: concurrent status updates converge to the latest timestamp, and a later update resurrects an element deleted by another replica. The AAL service path currently orders and fences these events rather than rejecting them semantically; that gap is a limitation in Section VII-C.

The comparison is well-defined on field-edit propagation, no-partition write acceptance, and the CRDT-LWW conflict cases. Infrastructure is asymmetric: the CRDT baseline is in memory, has no background anti-entropy, and converges only through harness-triggered *full_sync()* calls, so the relabelled row is post-quiescence forced sync, not live network impairment. AAL carries a durable Postgres outbox. These asymmetries favor CRDT where it leads; they are retained because the test is whether mergeable-by-default semantics can encode authority.

F. Backhaul recovers within about a second of WAN heal

V-B through V-E cover paths with correctness choices. V-F covers support-path backhaul: command-to-core forwarding after WAN heal. The partition is a surgical *tc* cut of the command-to-core path only; command Postgres and responder reachability remain available, so the measurement isolates backhaul flush time. Median recovery is 0.75, 0.91, and 0.23 s for 1, 10, and 60 s partitions, with p95 about one second (0.92,

1.04, and 1.05 s). Recovery is bounded by the forward poll loop and backlog, not outage length.

Together, V-B shows the authority boundary holds across transitions; V-C shows responder isolation cannot delay the journal; V-D shows field submissions cross the boundary durably and idempotently; V-E shows mergeability does not encode R3; and V-F shows support-path recovery is about one second. The cost remains where §IV places it: no automatic failover and lower write availability for authority-bearing operations, concentrated at operator-driven command transitions.

VII. SCOPE AND DISCUSSION

A. Why Single Writer Instead of CRDTs

Local-first research makes a strong case for replica autonomy and conflict-free merging in collaborative systems [16], [21]–[23]. That logic does not by itself satisfy R3. Incident state contains command decisions, ordered status transitions, and resource assignments for which CRDT-based merge has no defensible application semantics: there is no commutative merge function for a status transition that two responders attempt to drive in different directions, and any resolution rule that picks one of them implicitly invents the authority that R3 requires be explicit. The single-writer design adopted here makes that authority operational rather than emergent, at the cost of write availability at responder nodes during partition.

B. Threat Model and Security Scope

The target deployment assumes an organizational CA or identity provider, enrolled devices and users, and backend trust in certificate-derived identity only when the reverse proxy and service boundary are correctly configured. It handles replayed field submissions through *client_event_id* idempotency and limits tablet reachability through local edge APIs and network segmentation.

Several threats remain outside the prototype evaluation. A stolen or compromised tablet can be revoked only when nodes regain contact with the authority infrastructure or receive an updated trust bundle; instantaneous revocation during total disconnection is not provided. A stolen vehicle node is more serious because it may hold cached data and, if it is the command edge, operational authority; the architecture relies on device encryption, operational fencing, and audit rather than Byzantine tolerance. The design also does not address a compromised organizational CA, Byzantine command behavior, physical tamper resistance beyond deployment controls, or audit-log integrity against a fully compromised node. These are deployment and hardening questions, not properties established by the emulation.

C. Limitations and Scope

The CRDT-LWW comparison measures both systems on the executable service path where the comparison is well-defined: propagation latency, no-partition write acceptance, and post-quiescence convergence after a fixed outage duration. The two outage scenarios are not mechanically identical:

TABLE V

COMPARISON WITH A HANSSEN-STYLE OPERATION-BASED CRDT-LWW BASELINE. CRDT MEASUREMENTS ARE REAL RUNS OF THE BASELINE IMPLEMENTATION; AAL ENTRIES USE THE CURRENT SERVICE-PATH MEASUREMENTS AND EXPLICITLY MARK SEMANTIC GUARDS NOT IMPLEMENTED IN THE MEASURED PROTOTYPE. THE RELABELLED CONVERGENCE ROW REPORTS POST-QUIESCENCE CONVERGENCE AFTER HARNESS-TRIGGERED ANTI-ENTROPY, NOT LIVE NETWORK-PARTITION IMPAIRMENT.

Scenario	AAL result	CRDT-LWW result	Interpretation
Field-edit propagation	AAL bridge median 639.4 ms at qd=10; 931.5 ms at qd=100	CRDT qd=10 stressed/degraded 814.1 ms/2.54 s; qd=100 degraded 3.97 s	Both preserve durable propagation; CRDT anti-entropy is not an authority boundary.
Concurrent status edit	Current AAL service path sequences unique events; semantic rejection is model-level/not implemented in the measured prototype	30/30 LWW runs converge; winner counts contained: 30; median 12.8 ms	LWW accepts both writes and materializes only the latest value; losing op is only recoverable from the op log.
Delete/update race	Current AAL service path has no delete/update semantic guard; limitation disclosed	30/30 runs resurrect the element; median convergence 12.7 ms	Matches Hanssen’s stated LWW delete-semantics caveat.
No-partition throughput	AAL command-journal commit rate median 356.3 events/s (single writer)	CRDT aggregate local accept rate across three writers median 349.2 events/s	Different units; reports write-availability trade-off, not direct speedup.
Post-quiescence convergence (60 s outage, $n = 10$)	AAL command-to-core support-path recovery median 227.0 ms after a real tc WAN cut ($n = 30$)	CRDT all-replica convergence median 45.4 ms after anti-entropy resumes (no link impairment during the wait)	Comparable in outage duration, not transport mechanism; AAL keeps command-local ordering, CRDT converges once anti-entropy is exchanged.

the AAL WAN outage is a real tc cut of the command-to-core path, whereas the CRDT baseline exchanges operations only through harness-mediated anti-entropy, so its outage is modelled by withholding that exchange for the same duration rather than by impairing a live link. The convergence figures are therefore comparable in outage duration, not in transport mechanism. The absence of background gossip and live link impairment favors the CRDT baseline on this row; we retain that asymmetry because the comparison asks whether mergeable-by-default semantics can encode authority even when the baseline is given the easier infrastructure. For the two invariant-violation scenarios (concurrent authoritative edit and delete-resurrection), the CRDT result is measured on its service path while the AAL guarantee rests on the strict TLA+ model (§VI-B), whose medium configuration checks all six safety invariants over 681,092 states. Four of the six—*EpochAuthorityCoupling*, *NoForkedJournal*, *Durability-AcrossPromotion*, and *LocalIdempotency*—also have service-path witnesses in the V-B live runs; the remaining two, *SingleAuthority* and *SingleCommand*, are model-only because the promotion harness is single-edge. The AAL service path sequences incident-journal entries and enforces epoch fencing but does not yet implement semantic rejection for those two application-level conflicts, which is future work.

The throughput comparison has a unit boundary. CRDT-LWW reports aggregate local acceptance across replicas, which is the availability benefit of the design. AAL reports 356.3 events/s through the direct single incident-journal linearization point and 159.4 events/s end-to-end through the responder outbox and forwarding path. We therefore use the throughput result as a trade-off indicator, not as a direct apples-to-apples speed comparison.

The prototype evaluation runs on a single-host Docker Compose emulation with tc -netem impairment. This isolates protocol behavior and keeps the artifact reproducible, but it does not claim physical Rajant mesh behavior, measured WireGuard or mTLS overhead, or end-to-end Android validation.

The workload is single-incident and limited-scale. Multi-host emulation and a vehicle-mounted mesh deployment are future work.

Promotion is operator-initiated. The paper does not claim automatic failover or storage-layer leader election after silent command death; an operator runbook is part of the operational procedure. The service-path promotion check is also single-edge: it shows that stale-epoch traffic cannot poison the current command journal after an epoch change, not that a full multi-edge promotion has been measured under all operator races. End-to-end promotion wall-clock with a live operator-confirm step is out of scope for the reported emulation; only the protocol-side cost is measured (§VI-B), and the operator-confirm floor is asserted by design rather than measured.

The threat model is crash-fault, not Byzantine. A captured command vehicle or buggy command-edge service can poison the incident journal while it holds authority; this is a central trade-off of making command authority explicit. Deployment controls such as device encryption, key revocation, vehicle authentication, and audit replay are mitigation hooks, not properties established by this prototype.

VIII. CONCLUSION

Authority-aligned linearization rests on an operational fact that consensus and CRDT designs cannot assume: at every moment, one principal is designated as authoritative. The incident-journal linearization point is the command edge, the vehicle edge node that currently holds that authority. Reference data, field submissions, materialized state, and audit forwarding sit outside that boundary on weaker contracts. Command transfer uses fenced promotion rather than leader election.

The model checks and the AAL service-path prototype agree on the safety boundary. Under strict fencing, no two writers and no forked journal; the weak variant produces the expected split-brain witness. Under a responder partition, the command edge committed 20/20 writes. The CRDT-LWW comparison shows the trade-off: CRDT replicas accept local

writes at multiple nodes and converge quickly after anti-entropy exchange, while AAL preserves one incident journal and treats semantic conflict rejection as part of the linearization boundary. The cost is no automatic failover and lower write availability for authority-bearing operations.

Deploy AAL on a vehicle-mounted mesh, exercise fenced promotion across multiple command edges, implement service-path semantic rejection for authoritative conflicts, and add crash-boundary and Android end-to-end tests. The harder comparison is empirical, not architectural: measure the design against CoNICE-class consensus and CRDT-based emergency systems on workloads where command authority is contested rather than designated.

REFERENCES

- [1] M. Wiśniewski, K. Szwarc, and W. Skomra, “Continuity of essential services as an emerging challenge for societal resilience,” *IEEE Access*, vol. 11, pp. 44 614–44 635, 2023.
- [2] OECD, “Digital security and resilience in critical infrastructure and essential services,” OECD Digital Economy Papers, Paris, 2019.
- [3] B. Karaman, I. Bastürk, S. Taskin, E. Zeydan, F. Kara, E. A. Beyazit, M. Camelo, E. Björnson, and H. Yanikomeroglu, “Solutions for sustainable and resilient communication infrastructure in disaster relief and management scenarios,” *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 716–760, 2026.
- [4] E. G. Zimbelman, A. T. Hudak *et al.*, “Lost in the woods: Forest vegetation, and not topography, most affects the connectivity of mesh radio networks for public safety,” *PLOS ONE*, 2022.
- [5] Q. Wang, S. Wang *et al.*, “An overview of emergency communication networks,” *Remote Sensing*, 2023.
- [6] N. Andreassen, O. J. Borch, E. Ikonen *et al.*, “Information sharing and emergency response coordination,” *Safety Science*, 2020.
- [7] A. Marshall, C.-A. Wilson, and A. P. Dale, “Telecommunications and natural disasters in rural Australia: The role of digital capability in building disaster resilience,” *Journal of Rural Studies*, vol. 100, p. 102996, 2023.
- [8] M. Murphy, “Digital transformation for crisis volunteerism: A study in the aftermath of the swedish forest fires crisis in 2018,” Licentiate thesis, Linköping University, 2021. [Online]. Available: <https://www.diva-portal.org/smash/get/diva2:1570107/FULLTEXT01.pdf>
- [9] D. K. Allen, S. Karanasios, and A. P. Norman, “Information sharing and interoperability: The case of major incident management,” *European Journal of Information Systems*, vol. 23, no. 4, pp. 418–432, 2014.
- [10] Räddningstjänsten Storgöteborg, “Råd och anvisning 201: Insatsplanering vid farlig verksamhet,” Operational guidance document, 2023, revised 2023-10-25; valid from 2018-12-01. [Online]. Available: <https://www.rsgbg.se/globalassets/dokument/rad-och-anvisningar/rad-och-anvisning-201---insatsplanering-vid-farlig-verksamhet.pdf>
- [11] —, “Kommunal Plan för räddningsinsatser – Del I: Allmän del,” Municipal rescue-services plan, 2025, dated 2019-01-02; revised 2025-06-09. [Online]. Available: <https://www.rsgbg.se/globalassets/dokument/foretag-och-organisationer/seveso/rsg-kommunal-plan-for-raedningsinsats-del-i---allman-del.pdf>
- [12] S. Pilemalm, D. Andersson, and K. Yousefi Mojir, “Enabling organizational learning from rescue operations: The Swedish rescue services incident reporting system,” *International Journal of Emergency Services*, vol. 3, no. 2, pp. 101–117, 2014.
- [13] A. Westman *et al.*, “Mobilisation of emergency services for chemical incidents in Sweden – a multi-agency focus group study,” *Scandinavian Journal of Trauma, Resuscitation and Emergency Medicine*, vol. 29, p. 99, 2021.
- [14] C. Richardson, *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.
- [15] P. Helland, “Life beyond distributed transactions: An apostate’s opinion,” *Communications of the ACM*, vol. 60, no. 2, pp. 46–54, 2017.
- [16] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-first software: You own your data, in spite of the cloud,” in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, 2019.
- [17] P. K. Paul *et al.*, “CRIMP: Here crisis mapping goes offline,” *Journal of Network and Computer Applications*, 2019.
- [18] M. Kleppmann, *Designing Data-Intensive Applications*. O’Reilly Media, 2017.
- [19] P. S. Almeida, A. Shoker, and C. Baquero, “Efficient state-based CRDTs by delta-mutation,” in *International Conference on Networked Systems (NETYS)*, 2014.
- [20] V. B. F. Gomes, M. Kleppmann, D. P. Mulligan, and A. R. Beresford, “Verifying strong eventual consistency in distributed systems,” *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 1–28, 2017.
- [21] J. Haas, V. Bales *et al.*, “Lore: A programming model for verifiably safe local-first software,” *ACM Transactions on Programming Languages and Systems*, 2023.
- [22] M. Köhler, G. Salvaneschi *et al.*, “Consistent local-first software: Enforcing safety and invariants for local-first applications,” *IEEE Transactions on Software Engineering*, 2025.
- [23] K. Jannes, W. De Meuter *et al.*, “Owbesync: Seamless synchronization of distributed web clients,” *IEEE Transactions on Parallel and Distributed Systems*, 2021.
- [24] N. V. Lewchenko, G. Kaki, and B.-Y. E. Chang, “Bolt-on strong consistency: Specification, implementation, and verification,” *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA, pp. 1604–1631, 2025.
- [25] Ø. Hanssen, “Data replication for distributed emergency management systems,” in *Norsk IKT-konferanse for forskning og utdanning (NIK)*, 2025.
- [26] L. Lamport, “The part-time parliament,” *ACM Transactions on Computer Systems*, vol. 16, no. 2, pp. 133–169, 1998.
- [27] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *Proceedings of the 2014 USENIX Annual Technical Conference*, 2014, pp. 305–320.
- [28] M. Balakrishnan, D. Malkhi, T. Wobber *et al.*, “Tango: Distributed data structures over a shared log,” in *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, 2013, pp. 325–340.
- [29] A. Katsarakis, V. Gavrielatos, M. R. S. Katebzadeh *et al.*, “Hermes: A fast, fault-tolerant and linearizable replication protocol,” in *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.
- [30] Z. Liang *et al.*, “HoliPaxos: Towards more predictable performance in state machine replication,” *Proceedings of the VLDB Endowment*, 2025.
- [31] A. Gooding *et al.*, “LARK: Linearizability algorithms for replicated keys in aerospace,” *arXiv preprint*, 2025.
- [32] R. van Renesse and F. B. Schneider, “Chain replication for supporting high throughput and availability,” in *6th Symposium on Operating Systems Design and Implementation (OSDI 04)*. San Francisco, CA: USENIX Association, Dec. 2004, pp. 91–104. [Online]. Available: <https://www.usenix.org/conference/osdi-04/chain-replication-supporting-high-throughput-and-availability>
- [33] M. Jahanian and K. K. Ramakrishnan, “CoNICE: Consensus in intermittently-connected environments by exploiting naming with application to emergency response,” *IEEE/ACM Transactions on Networking*, vol. 30, no. 4, pp. 1926–1939, 2022.
- [34] A. Kumbhar, F. Koohifar, I. Guvenc, and B. Mueller, “A survey on legacy and emerging technologies for public safety communications,” *IEEE Communications Surveys & Tutorials*, 2015.
- [35] M. F. H. Sagor, A. Haroon, R. Stoleru, S. Bhunia, A. Altaaweel, M. Chao, L. Jin, M. Maurice, and R. Blalock, “DistressNet-NG: A resilient data storage and sharing framework for mobile edge computing in cyber-physical systems,” *ACM Transactions on Cyber-Physical Systems*, vol. 8, no. 3, pp. 37:1–37:31, 2024.
- [36] M. Aboualola, K. Abualsaud, T. Khattab, N. Zorba, and H. Hassanein, “Edge technologies for disaster management: A survey of social media and artificial intelligence integration,” *IEEE Access*, vol. 11, pp. 73 782–73 802, 2023.
- [37] C. Li, D. Porto, A. Clement, J. Gehrke, N. Preguiça, and R. Rodrigues, “Making geo-replicated systems fast as possible, consistent when necessary,” in *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*. Hollywood, CA: USENIX Association, 2012, pp. 265–278. [Online]. Available: <https://www.usenix.org/conference/osdi12/technical-sessions/presentation/li>